

# Guião 03 – Virtualização com KVM e LXC no Proxmox

Infraestruturas de Computação na Nuvem, Aula 03

---

## 1 Objetivos

---

- Criar e gerir máquinas virtuais KVM por linha de comandos, com a ferramenta `qm` do Proxmox VE.
- Criar e gerir um contentor de sistema LXC com a ferramenta `pct`, e comparar com a VM equivalente.
- Observar na prática as diferenças de arranque, consumo de memória e isolamento entre KVM e LXC.
- Criar um snapshot e um clone, e converter uma VM em template.
- Medir, de forma básica, o overhead de virtualização com um benchmark simples.

## 2 Pré-requisitos e Material

---

- Acesso ao Proxmox VE do servidor central, com as credenciais do Guião 01.
- A VM `icn-lab` criada no Guião 01.
- Intervalo de VMIDs atribuído pela docente (neste guião usamos `<vmid>` como marcador – substituir pelos números efetivamente atribuídos).
- Acesso por SSH ao *node* Proxmox para os comandos `qm/pct`. Se o acesso à shell do node não estiver disponível para os alunos, todos os passos têm equivalente na interface web, e a componente CLI é demonstrada pela docente.

## 3 Introdução

---

Na aula teórica vimos que existem dois modelos distintos de virtualização: a **virtualização completa** (uma VM com o seu próprio kernel, sobre um hypervisor) e a **virtualização ao nível do sistema operativo** (contentores que partilham o kernel do host). O Proxmox VE é particularmente útil para estudar esta distinção porque implementa ambos os modelos na mesma plataforma: **KVM** para máquinas virtuais e **LXC** para contentores de sistema.

Neste guião vamos criar um exemplar de cada um, lado a lado, e medir concretamente as diferenças. Usaremos sobretudo a linha de comandos (`qm` e `pct`), porque é a forma como um administrador automatiza estas operações em escala – uma competência que será retomada na Aula 07, quando automatizarmos este mesmo tipo de provisionamento com Ansible e Terraform.

## 4 Parte 1 – Máquinas Virtuais KVM com `qm`

---

1. Listar as VMs existentes e inspecionar a sua configuração.

```
$ qm list
$ qm config <vmid-da-icn-lab>
```

Identifique na configuração: a memória atribuída, o número de cores, o disco (e o seu formato) e a interface de rede.

2. **Consultar o estado e os recursos em uso.**

```
$ qm status <vmid-da-icn-lab> --verbose
```

3. **Criar uma nova VM apenas por linha de comandos.** Repare que cada opção corresponde a um dos recursos virtualizados estudados na teoria:

```
$ qm create <novo-vmid> \  
  --name icn-lab-cli \  
  --memory 2048 \  
  --cores 2 \  
  --net0 virtio,bridge=vmbr0 \  
  --scsihw virtio-scsi-pci \  
  --ostype l26
```

A opção `--net0 virtio` usa a interface paravirtualizada *virtio* referida na aula, em vez de emular uma placa de rede real.

4. **Adicionar um disco à nova VM.** O disco é criado no storage indicado (confirmar o nome do storage disponível com `pvesm status`):

```
$ pvesm status  
$ qm set <novo-vmid> --scsi0 <storage>:20
```

Isto cria um disco de 20 GB. Consulte a configuração resultante e identifique o formato usado:

```
$ qm config <novo-vmid>
```

5. **Comparar thin provisioning com o tamanho nominal.** Verifique quanto espaço o disco ocupa realmente no armazenamento, em contraste com os 20 GB declarados:

```
$ pvesm list <storage>
```

Registe a diferença entre o tamanho nominal e o espaço efetivamente alocado.

## 5 Parte 2 – Contentores de Sistema LXC com pct

6. **Listar os templates de contentor disponíveis.**

```
$ pveam available | grep ubuntu  
$ pveam list <storage-de-templates>
```

Se o template pretendido ainda não estiver descarregado no servidor, a docente indicará qual usar (o download só é feito uma vez, para toda a turma).

7. **Criar um contentor LXC.** Note como o comando é significativamente mais simples do que a criação da VM: não há ISO nem instalação de sistema operativo, porque o template já contém o sistema de ficheiros pronto:

```
$ pct create <novo-ctid> \  
  <storage-de-templates>:vztmpl/<nome-do-template>.tar.zst \  
  --hostname icn-ct \  
  --memory 2048 \  
  --
```

```
--cores 2 \  
--net0 name=eth0,bridge=vmbr0,ip=dhcp \  
--rootfs <storage>:8
```

8. **Arrancar o contentor e cronometrar o arranque.** Compare com o tempo de arranque de uma VM completa:

```
$ time pct start <novo-ctid>  
$ pct status <novo-ctid>
```

9. **Entrar no contentor e inspecionar o kernel.** Este é o passo mais revelador do guião:

```
$ pct enter <novo-ctid>  
root@icn-ct:~# uname -r  
root@icn-ct:~# cat /etc/os-release  
root@icn-ct:~# exit
```

Compare agora com o kernel visto *dentro da VM icn-lab* (via SSH) e com o kernel do próprio *node Proxmox*:

```
# no node Proxmox:  
$ uname -r
```

Registe as três versões de kernel observadas. A relação entre elas é o ponto central desta aula.

10. **Comparar o consumo de memória.** Observe quanto ocupa efetivamente cada um:

```
$ pct status <novo-ctid> --verbose  
$ qm status <vmid-da-icn-lab> --verbose
```

## 6 Parte 3 – Snapshots, Clones e Templates

11. **Criar um snapshot** da VM *icn-lab* antes de qualquer alteração:

```
$ qm snapshot <vmid-da-icn-lab> estado-inicial \  
--description "Antes das experiencias da Aula 03"  
$ qm listsnapshot <vmid-da-icn-lab>
```

12. **Provocar uma alteração e reverter.** Dentro da VM (via SSH), crie um ficheiro reconhecível:

```
$ echo "alteracao de teste" > ~/teste-snapshot.txt
```

De volta ao node, reverta a VM para o snapshot e confirme que o ficheiro desapareceu:

```
$ qm rollback <vmid-da-icn-lab> estado-inicial  
$ qm start <vmid-da-icn-lab>
```

13. **Clonar a VM.** Compare os dois modos de clonagem discutidos na teoria:

```
# Clone completo (independente):  
$ qm clone <vmid-da-icn-lab> <novo-vmid-2> --name icn-lab-clone --full  
  
# Clone ligado (partilha as camadas base, ocupa menos espaco):  
$ qm clone <vmid-da-icn-lab> <novo-vmid-3> --name icn-lab-linked
```

Verifique o espaço ocupado por cada um com `pvesm list <storage>` e registre a diferença. *Nota:* o clone ligado só é suportado em alguns tipos de storage e exige que a origem seja um template; se o comando falhar, registre a mensagem de erro obtida e explique-a com base no que sabe sobre linked clones.

14. **Converter uma VM em template.** Uma vez convertida, a VM passa a ser só de leitura e serve de base para novas máquinas:

```
$ qm template <novo-vmid>
$ qm list
```

## 7 Parte 4 – Medir o Overhead de Virtualização

---

15. **Executar um benchmark de CPU** dentro da VM KVM (via SSH) e dentro do contentor LXC (via `pct enter`), com parâmetros idênticos:

```
$ sudo apt update && sudo apt install sysbench -y
$ sysbench cpu --cpu-max-prime=20000 --threads=2 run
```

Registe o valor de *events per second* em cada ambiente.

16. **Comparar com o desempenho no próprio node** (se tiver acesso à shell do node e a docente o autorizar; caso contrário, use o valor de referência que ela fornecer):

```
$ sysbench cpu --cpu-max-prime=20000 --threads=2 run
```

Construa uma pequena tabela com os três valores (node, VM KVM, contentor LXC) e calcule a diferença percentual face ao node.

17. **Limpeza.** No final da aula, remova os recursos criados para não esgotar a capacidade do servidor partilhado:

```
$ qm stop <novo-vmid-2>; qm destroy <novo-vmid-2>
$ pct stop <novo-ctid>; pct destroy <novo-ctid>
```

Mantenha a VM `icn-lab` original, que continuará a ser usada nos guiões seguintes.

## 8 Entregável / Questões de Verificação

---

- a) Submeta a tabela com os três valores de `sysbench` (node, VM KVM, contentor LXC) e uma breve interpretação das diferenças observadas.
- b) Registe as três versões de kernel observadas (node, VM, contentor). Porque é que a do contentor coincide com a do node e a da VM não? Relacione com a distinção entre virtualização completa e virtualização ao nível do SO.
- c) Compare o tempo de arranque e o consumo de memória do contentor LXC e da VM KVM. Que explicação arquitetural justifica a diferença?
- d) Que diferença observou, em espaço ocupado, entre o tamanho nominal do disco (20 GB) e o espaço efetivamente alocado? Que mecanismo explica isto e que risco operacional introduz?
- e) Em que situação escolheria um contentor LXC em vez de uma VM KVM, e em que situação faria o contrário? Dê um exemplo concreto para cada caso.

- f) Qual a diferença prática entre um snapshot, um clone completo e um template? Indique um cenário de administração real para cada um.